React.js: Components, State, and UI Architecture

A practical introduction to building maintainable user interfaces

Purpose: This document is a fictional, self-contained learning resource created for experimenting with PDF text extraction, chunking, embeddings, retrieval, and question answering in a Retrieval-Augmented Generation (RAG) pipeline.

Contents

1. What is React?

2. Components and props

3. State and event handling

4. Effects and data fetching

5. Forms, lists, and keys

6. Performance and maintainability

7. Testing and accessibility

1. What is React?

React is a JavaScript library for building user interfaces from reusable components. A component describes a part of the interface and returns a tree of elements that React can render. React is commonly used for single-page applications, dashboards, forms, and interactive product interfaces.

React focuses on the view layer. Routing, server communication, global state, and build tooling are typically handled by additional libraries or platform features. A React application can be created with a modern build tool or integrated into an existing application.

A minimal function component

```
function Welcome({ name }) {
  return &lt;h1&gt;Hello, {name}!&lt;/h1&gt;;
}

export default Welcome;
```

2. Components and props

Components are reusable units of UI. Component names conventionally begin with a capital letter. Props are read-only inputs passed from a parent component to a child component. They help a component render different content without duplicating its implementation.

Keep components focused on a clear responsibility. Extract repeated UI into reusable components, and pass only the data and callbacks a child needs.

Passing props

```
function UserCard({ user }) {
  return (
    &lt;section&gt;
      &lt;h2&gt;{user.name}&lt;/h2&gt;
      &lt;p&gt;{user.role}&lt;/p&gt;
    &lt;/section&gt;
  );
}

&lt;UserCard user={{ name: "Asha", role: "Developer" }} /&gt;
```

3. State and event handling

State represents data that can change while a component is in use. The useState Hook returns the current state value and a setter function. Calling the setter schedules a re-render with the updated value. State should be updated immutably when it contains objects or arrays.

Event handlers respond to user actions such as clicks, typing, and form submission. Pass a function to an event prop instead of calling it during rendering.

A counter using state

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    &lt;button onClick={() =&gt; setCount(c =&gt; c + 1)}&gt;
      Count: {count}
    &lt;/button&gt;
  );
}
```

4. Effects and data fetching

useEffect synchronizes a component with an external system, such as a browser API, subscription, or network request. Its dependency array determines when the effect is re-run. Effects should include cleanup when a subscription or other resource must be released.

For production data fetching, consider loading and error states, request cancellation or stale-response handling, caching, retries where appropriate, and avoiding duplicate requests. A server-state library can provide these features more consistently than hand-written effects.

A basic effect

```
import { useEffect, useState } from "react";

function Status() {
  const [status, setStatus] = useState("Loading");

  useEffect(() =&gt; {
    let active = true;
    fetch("/api/status")
      .then(response =&gt; {
        if (!response.ok) throw new Error("Request failed");
        return response.json();
      })
      .then(data =&gt; {
        if (active) setStatus(data.status);
      })
      .catch(() =&gt; {
        if (active) setStatus("Unavailable");
      });
    return () =&gt; { active = false; };
  }, []);

  return &lt;p&gt;{status}&lt;/p&gt;;
}
```

5. Forms, lists, and keys

A controlled input receives its value from React state and updates that state through an onChange handler. Controlled forms make validation and conditional UI easier to reason about. For larger forms, a dedicated form library may reduce repetitive code.

When rendering a list with map, give each sibling a stable key, usually a database identifier. Keys help React associate rendered elements with the correct items when a list changes. Array indexes are often a poor choice for lists that can be reordered or edited.

Rendering a list

```
const tasks = [
  { id: "t1", title: "Read the docs" },
  { id: "t2", title: "Write a component" }
];

function TaskList() {
  return (
    &lt;ul&gt;
      {tasks.map(task =&gt; (
        &lt;li key={task.id}&gt;{task.title}&lt;/li&gt;
      ))}
    &lt;/ul&gt;
  );
}
```

6. Performance and maintainability

Start performance work by measuring. Unnecessary re-renders, expensive calculations, large bundles, and excessive DOM nodes can affect responsiveness. React.memo, useMemo, and useCallback can help in specific cases, but they are not substitutes for measurement or clear component design.

Split code by route or feature when useful, keep state close to where it is needed, and avoid storing values that can be calculated from existing props or state. Error boundaries can keep certain rendering errors from taking down an entire interface.

7. Testing and accessibility

Test visible behavior: users should be able to complete workflows, see validation feedback, and understand loading and error states. Unit tests can cover pure functions; component and integration tests can verify interactions; end-to-end tests can cover critical journeys.

Use semantic HTML, associate labels with form controls, provide keyboard access, preserve visible focus indicators, and ensure sufficient color contrast. Accessibility should be part of component design rather than a final patch.

Glossary

Component — A reusable unit that describes part of a user interface.

Props — Read-only inputs supplied to a component.

State — Data managed by a component that can change over time.

Hook — A React function such as useState or useEffect used to access React features.

Effect — A synchronization mechanism for work outside the render calculation.

Key — A stable identifier used when rendering collections of elements.

Controlled input — A form control whose value is driven by React state.

Practice questions for retrieval

Use these questions to test whether your RAG pipeline retrieves the relevant passages rather than answering from general model knowledge.

1. What is React, and which concerns are usually handled by additional libraries?
2. How are props different from state?
3. What does the useState Hook return?
4. When is useEffect appropriate, and why might cleanup be necessary?
5. Why should list items have stable keys?
6. Name three accessibility practices for React forms.
7. Why should developers measure before applying memoization?

End of sample document.